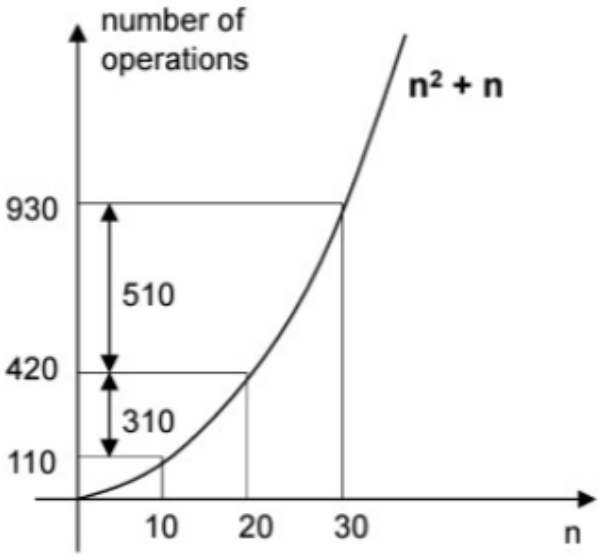
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10 / 6 / 26	**DATE SUBMITTED:**	10 / 13 /26
NAME:	Fabonan, Kurt Lorenz C. Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 11.1: Basic Algorithm Analysis

1. PROCEDURE OUTPUT

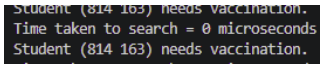
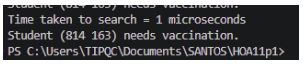


n (size of array)	number of operations
10	110
20	420
30	930

Analysis: The order of operations increases relatively to the size of the array due to its complexity. This results at slope curving upwards to the x and y axis as the given results is the array having the space complexity of $n^2 + n$.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 microseconds	Time taken to search = 0 microseconds Student (814 163) needs vaccination.	It is done faster due to it having the smaller the smaller int size.

10000	0 microseconds		It takes the same time as the first one but it slowly increases its complexity to less than a microsecond.
100000	1 microseconds		It takes the longest time and has shown an increase in a microsecond due to its max size being increased drastically.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis.

Problem 1: Consider a program that returns true if the elements in an array are unique.

Theoretical Analysis:

Fabonan - Due to i and j being integers, their size is 4. Array sizes are almost always integers, so the size is 4. So, their values are combined together to be 8. An integer array is multiplied by the number of elements inside it, so it is 4n where n is the number of elements inside the array. So, the total size is i + j + array, so the size complexity is S(n) = 4n + 8. In terms of time complexity, it is T(n) = n^2 because the algorithm is a nested for loop, because the max operations in the outer for loop is the number of operations in the inner loop, which is in every operation of every outer loop.

Santos - The space complexity of the array is S(n) = 4n + 8 since i and j and is equal to 4 while the array stores an integer value, the max size of the array will be multiplied by four then added by i and j as 8. The time complexity of the code is T(n) = n^2 as the worst case while T(n) = 1 as the best case. If both values are not the same on the first two elements, then the best case will likely happen while if the values are all the same, then the worst case will happen.

Experimental Analysis:

Fabonan

Code:

```cpp
#include <iostream>
#include <cstdlib>

bool unique(int A[]);

const int max_size = 10;

int main(void) {

    int arr[max_size];

    for (int a = 0; a < max_size; a++) {

        arr[a] = rand();
    }

    std::cout << "The numbers in the array are: ";
    for (int p = 0; p < max_size; p++) {

        std::cout << arr[p] << ", ";
    }

    std::cout << "\n" << std::endl;

    unique(arr);


}

bool unique(int A[]) {

    for (int i = 0; i < max_size; i++) {

        for (int j = i + 1; j < max_size; j++){

            if (A[i] == A[j]) {

                std::cout << "All of the elements in the array are not unique" << std::endl;

                return false;
            }
        }
    }

    std::cout << "All of the elements in the array are unique" << std::endl;

    return true;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\CPE21S3_FABONAN> cd "c:\Users\TIPQC\Documents\CPE21S3_FABONAN\Basic_Algo_Analysis\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
The numbers in the array are: 41, 18467, 6334, 26500, 19169, 15724, 11478, 29358, 26962, 24464,

All of the elements in the array are unique
PS C:\Users\TIPQC\Documents\CPE21S3_FABONAN\Basic_Algo_Analysis>
```

Santos

Code:

```cpp
#include <iostream>
#include <time.h>

int main(){
    // Generate random values (Assume that 100 is n for space complexity)
    int a[100];
    srand(time(0));
    for(int i = 0; i < 100; i++){
        a[i] = rand();
    }

    // Time complexity here:
    for(int i = 0; i < 100; i++){
        for(int j = i+1; j < 100; j++){
            if (a[i] == a[j]){
            std::cout << "Returned False \n"; // If both values stays the same, then worst case.
            return false;
            }
        }
    }
    std::cout << "Returned True \n"; // If only return true is printed, then best case.
    return true;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\SANTOS\HOA11p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA11p1\" ; if ($?) { g++ Supplementary.cpp -o S
upplementary } ; if ($?) { .\Supplementary }
Returned True
```

Analysis and Comparison:

Fabonan - The code operation is exactly the same as the pseudocode and as such, it concludes with the time complexities S(n) = 4n + 8 and T(n) = n^2.

Santos - Both of my analyses have the same outcome and have the same complexity output of both time and space complexity. They both concluded with the space complexity of S(n) = 4n + 8 and a time complexity of T(n) = n^2.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

Theoretical Analysis:

Fabonan - The time complexity of the recursive power is T(n) = n because the recursion is done by 1 operation which simply subtracts the operation, while the binary recursive power is T(n) = log n due to how it has a division in the algorithm, where log n basically means how much can

you divide the n until it reaches 0 or 1. The size complexity is S(n) = 8 x (n + 1) or S(n) 8n + 8 for normal recursion, and S(n) = 12 x (log n + 1) or 12 log n + 12 for binary recursive.

Santos - The time complexity of the recursive power is T(n) = n while the time complexity of the binary recursive power is T(n) = log n. Since both are run inside main, time complexity takes the highest value, making the combined time complexity as T(n) = n. As for the space complexity, the recursive power is S(n) = n. While the binary recursive power is S(n) = log n. It has the combined time complexity of S(n) = 8n + 16 as it counts both x and n variables that are declared in the main function while also taking into account the recursive power function and the binary recursive power function.

Experimental Analysis:

Fabonan

Code:

Basic_Algo_Analysis > bain.cpp > main(void)

```cpp
#include <iostream>

int Rpower(int w, int x);
int Brpower(int y, int z);

int main(void) {


    int Base = 0, Power = 0;

    std::cout << "Base number:";
    std::cin >> Base;

    std::cout << "\n\nPower number: ";
    std::cin >> Power;

    std::cout << "The Rpower is: " << Rpower(Base, Power) << ", while the Brpower is: " << Brpower(Base, Power) << std::endl;

    return 0;

}

int Rpower(int w, int x){

    if (x == 0) return 1;

    return w * Rpower (w, x - 1);
}

int Brpower(int y, int z){

    if (z == 0) return 1;

    int A = 0;

    if (z % 2 == 1) {

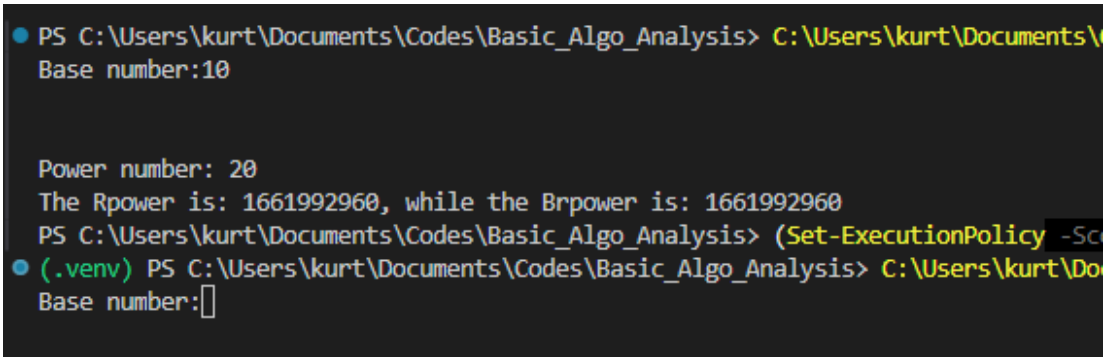
        A = Brpower(y, (z-1)/2);
        return y*A*A;

    } else {

        A = Brpower (y, z/2);

        return A*A;
    }
}
```

Output:


PS C:\Users\kurt\Documents\Codes\Basic_Algo_Analysis> C:\Users\kurt\Documents\
Base number:10
Power number: 20
The Rpower is: 1661992960, while the Brpower is: 1661992960
PS C:\Users\kurt\Documents\Codes\Basic_Algo_Analysis> (Set-ExecutionPolicy -Sc
(.venv) PS C:\Users\kurt\Documents\Codes\Basic_Algo_Analysis> C:\Users\kurt\Do
Base number:

Santos

Code:

```cpp
#include <iostream>

// Time complexity: T(n) = O(n), Space complexity: S(n) = n
int rpower(int x, int n){
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}

// Time complexity: T(n) = O(log n), Space complexity: S(n) = log n
int brpower(int x, int n){
    if (n == 0) return 1;

    int y;
    if (n % 2 == 1){
        y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }
    else{
        y = brpower(x, n / 2);
        return y * y;
    }
}

int main(){
    int x = 2;
    int n = 10;

    // Time complexity: T(n) = O(n), Space complexity: S(n) = 8n + 16
    std::cout << "rpower: " << rpower(x, n) << "\n";
    std::cout << "brpower: " << brpower(x, n) << "\n";

    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\SANTOS\HOA11p1> cd "c:\Users\TIPQC\Documents\SANTOS\HOA11p1\" ; if ($?) { g++ Supplementary_B.
cpp -o Supplementary_B } ; if ($?) { .\Supplementary_B }
rpower: 1024
brpower: 1024
PS C:\Users\TIPQC\Documents\SANTOS\HOA11p1>
```

Analysis and Comparison:

Fabonan - The results are the same, but the internal workings are different. Binary would be much faster than normal recursive due to division shrinking the operations. However, due to both rpower and bpower being used in the same program, the time complexity of the program is T(n) = n. Size is combined as well where the size of recursive and binary recursive is combined. So, the S(n) = 8n + 8 of recursive and the S(n) of 12 log n + 12 is combined to 8n + 12 log n + 20.

Santos - The binary recursive frees up bytes from the stacks, resulting in a space complexity of S(n) = log n. With this, when the two functions are used below, they only have a total of S(n) = 8(n + 2) due to the variable declared in main and other variables being declared inside the functions, making the final space complexity as S(n) = 8n + 16. Since time complexity takes the highest time complexity when combining the two, the combined time complexity is T(n) = n.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.